

Trabajo Práctico Integrador

Universidad Técnica Nacional

Carrera: Tecnicatura Universitaria en Programación.

Materia: Bases de Datos I.

Proyecto: Base de Datos del Sistema de Gestión de Pedidos de Comida.

Alumno: Francisco Bultynch.

Fecha: 23/06/2026

Índice

1. Introducción.....	2
2. Modelo de datos.....	2
3. Implementación de la base de datos.....	3
4. Carga masiva de datos.....	3
5. Consulta e índices.....	6
6. Seguridad e integridad.....	8
7. Servicios implementados.....	11
8. Concurrencias y transacciones.....	12
9. Conclusiones.....	14
10. Bibliografía.....	14
11. Anexo IA.....	15

Introducción

En este trabajo práctico teníamos como objetivo diseñar e implementar una base de datos relacional para un sistema de gestión de pedidos de un local gastronómico denominado Food Store. El sistema permite administrar usuarios, categorías, productos y pedidos, manteniendo la integridad y consistencia de los datos. Para el desarrollo se utilizó MySQL Workbench y el motor de almacenamiento InnoDB. Se implementaron claves primarias, claves foráneas, restricciones de integridad, carga masiva de datos, consultas avanzadas, índices, mecanismos de seguridad y control de transacciones. Además, se realizaron pruebas de concurrencia y bloqueo con el fin de analizar el comportamiento del sistema ante operaciones simultáneas.

Modelo de datos

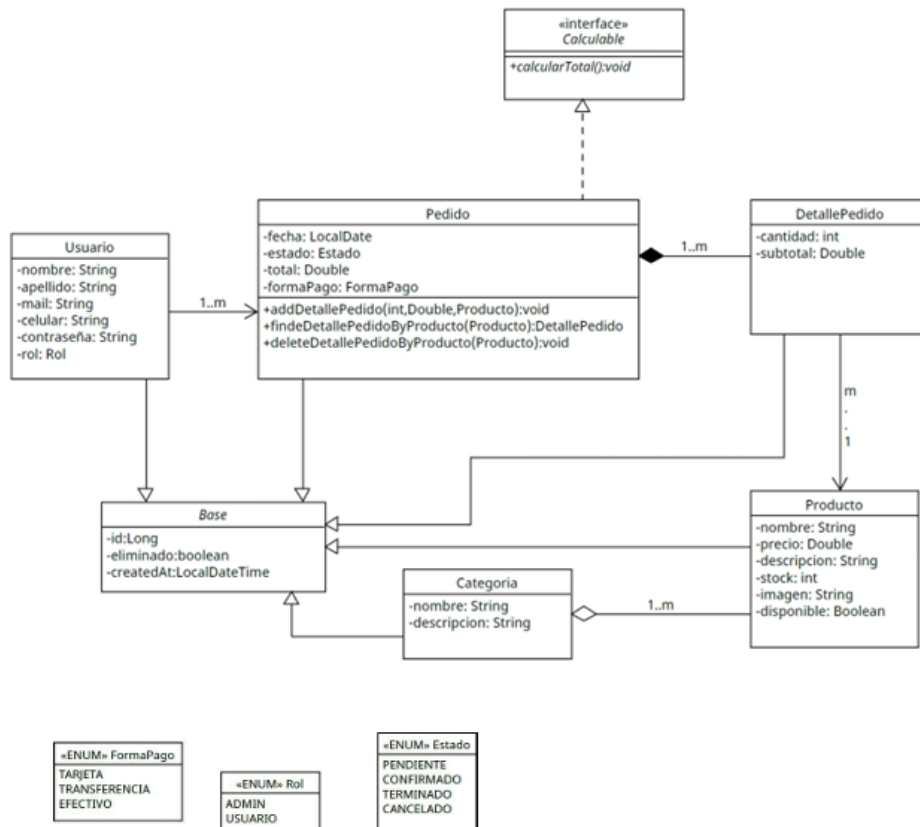
A partir del análisis del dominio se identificaron las siguientes entidades:

- Usuario
- Categoría.
- Producto.
- Pedido.
- DetallePedido.

Las relaciones implementadas fueron:

- Un usuario puede realizar varios pedidos.
- Un pedido puede contener uno o varios detalles.
- Cada detalle hace referencia a un producto.
- Cada producto pertenece a una categoría.

Se utilizó el modelo entidad-relación proporcionado como referencia para obtener posteriormente el modelo relacional implementado en MySQL.



Implementación de la base de datos

Se creó la base de datos denominada foodstore_db y se implementaron las tablas categoria, producto, usuario, pedido y detalle_pedido. Luego se definieron claves primarias para identificar de forma única cada registro y claves foráneas para mantener la integridad referencial entre las tablas. También se implementaron restricciones de integridad mediante:

- Restricciones UNIQUE para evitar duplicados.
- Restricciones CHECK para controlar valores inválidos.
- Restricciones NOT NULL para garantizar la obligatoriedad de ciertos atributos.

Posteriormente se realizaron inserciones válidas y prueba de inserciones inválidas para verificar el correcto funcionamiento de las restricciones.

Captura de pantalla de la creación de las tablas:

```

3 • CREATE TABLE categoria(
4     id BIGINT AUTO_INCREMENT,
5     nombre VARCHAR(100) NOT NULL,
6     descripcion VARCHAR(255),
7     eliminado BOOLEAN NOT NULL DEFAULT FALSE,
8     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
9
10    CONSTRAINT pk_categoria
11        PRIMARY KEY(id),
12
13    CONSTRAINT uk_categoria_nombre
14        UNIQUE(nombre)
15 );
16
17 • CREATE TABLE producto(
18     id BIGINT AUTO_INCREMENT,
19     nombre VARCHAR(100) NOT NULL,
20     precio DECIMAL(10,2) NOT NULL,
21     descripcion VARCHAR(255),
22     stock INT NOT NULL,
23     imagen VARCHAR(255),
24     disponible BOOLEAN NOT NULL DEFAULT TRUE,
25     categoria_id BIGINT NOT NULL,
26     eliminado BOOLEAN NOT NULL DEFAULT FALSE,
27     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
28
29    CONSTRAINT pk_producto
30        PRIMARY KEY(id),
31
32    CONSTRAINT fk_producto_categoria
33        FOREIGN KEY(categoria_id)
34        REFERENCES categoria(id),
35
36    CONSTRAINT chk_precio
37        CHECK(precio >= 0),
38
39    CONSTRAINT chk_stock
40        CHECK(stock >= 0)
41 );
42
43 • CREATE TABLE usuario(
44     id BIGINT AUTO_INCREMENT,
45     nombre VARCHAR(100) NOT NULL,
46     apellido VARCHAR(100) NOT NULL,
47     mail VARCHAR(150) NOT NULL,
48     celular VARCHAR(30),
49     contrasena VARCHAR(100) NOT NULL,
50     rol ENUM('ADMIN', 'USUARIO') NOT NULL,
51     eliminado BOOLEAN NOT NULL DEFAULT FALSE,
52     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
53
54    CONSTRAINT pk_usuario
55        PRIMARY KEY(id),
56
57    CONSTRAINT uk_mail
58        UNIQUE(mail)
59 );

```

```

61 • ○ CREATE TABLE pedido(
62     id BIGINT AUTO_INCREMENT,
63     fecha DATE NOT NULL,
64     estado ENUM(
65         'PENDIENTE',
66         'CONFIRMADO',
67         'TERMINADO',
68         'CANCELADO'
69     ) NOT NULL,
70
71     forma_pago ENUM(
72         'TARJETA',
73         'TRANSFERENCIA',
74         'EFECTIVO'
75     ) NOT NULL,
76
77     total DECIMAL(10,2) NOT NULL,
78     usuario_id BIGINT NOT NULL,
79     eliminado BOOLEAN NOT NULL DEFAULT FALSE,
80     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
81
82     CONSTRAINT pk_pedido
83         PRIMARY KEY(id),
84
85     CONSTRAINT fk_pedido_usuario
86         FOREIGN KEY(usuario_id)
87         REFERENCES usuario(id),
88
89     CONSTRAINT chk_total
90         CHECK(total >= 0)
91 );

```

```

93 • ○ CREATE TABLE detalle_pedido(
94     id BIGINT AUTO_INCREMENT,
95     cantidad INT NOT NULL,
96     subtotal DECIMAL(10,2) NOT NULL,
97     pedido_id BIGINT NOT NULL,
98     producto_id BIGINT NOT NULL,
99     eliminado BOOLEAN NOT NULL DEFAULT FALSE,
100     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
101
102     CONSTRAINT pk_detalle
103         PRIMARY KEY(id),
104
105     CONSTRAINT fk_detalle_pedido
106         FOREIGN KEY(pedido_id)
107         REFERENCES pedido(id),
108
109     CONSTRAINT fk_detalle_producto
110         FOREIGN KEY(producto_id)
111         REFERENCES producto(id),
112
113     CONSTRAINT chk_cantidad
114         CHECK(cantidad > 0),
115
116     CONSTRAINT chk_subtotal
117         CHECK(subtotal >= 0)
118 );

```

Carga masiva de datos

Con el objetivo de disponer de un volumen de información representativo, se realizó una carga masiva de datos sobre las tablas del sistema. Se generaron registros para categorías, productos, usuarios, pedidos y detalles de pedidos, superando los 10000 registros totales requeridos por la consigna.

Posteriormente se efectuaron verificaciones de consistencia para comprobar que no existieran registros huérfanos ni inconsistencias entre las relaciones implementadas. Además, se verificó que no existieran valores inválidos, como cantidades negativas o referencias a claves inexistentes.

```
129 • SELECT COUNT(*) AS categorias FROM categoria;
130
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
categorias			
▶	10		

```
131 • SELECT COUNT(*) AS productos FROM producto;
132
133 • SELECT COUNT(*) AS usuarios FROM usuario;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
productos			
▶	52		

```
133 • SELECT COUNT(*) AS usuarios FROM usuario;
134
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
usuarios			
▶	1002		

```
135 • SELECT COUNT(*) AS pedidos FROM pedido;
136
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
pedidos			
▶	2501		

```
137 • SELECT COUNT(*) AS detalles FROM detalle_pedido;
```

```
138
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	detalles
▶	2

```
139 -- VERIFICAMOS LAS FK HUÉRFANAS
```

```
140 -- pedidos sin usuarios
```

```
141
```

```
142 • SELECT COUNT(*) FROM pedido p LEFT JOIN usuario u ON p.usuario_id = u.id WHERE u.id IS NULL;
```

```
143
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	COUNT(*)
▶	0

```
144 -- detalles sin pedidos
```

```
145
```

```
146 • SELECT COUNT(*) FROM detalle_pedido d LEFT JOIN pedido p ON d.pedido_id = p.id WHERE p.id IS NULL;
```

```
147
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	COUNT(*)
▶	0

```
148 -- detalles sin producto
```

```
149
```

```
150 • SELECT COUNT(*) FROM detalle_pedido d LEFT JOIN producto p ON d.producto_id = p.id WHERE p.id IS NULL;
```

```
151
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	COUNT(*)
▶	0

```
152 -- verificamos el stock negativo
```

```
153
```

```
154 • SELECT * FROM producto WHERE stock < 0;
```

```
155
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	id	nombre	precio	descripcion	stock	imagen	disponible	categoria_id	eliminado	created_at
▶										

```
156 • SELECT * FROM producto LIMIT 20;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows: |

	id	nombre	precio	descripcion	stock	imagen	disponible	categoria_id	eliminado	created_at
▶	2	Hamburguesa Completa	12000.00	Con papas fritas	15	hamburguesa.jpg	1	2	0	2026-06-23 01:15:28
	3	Producto 1	8784.68	Producto generado automaticamente	88	producto.jpg	1	9	0	2026-06-23 01:32:08
	4	Producto 2	7673.56	Producto generado automaticamente	79	producto.jpg	1	6	0	2026-06-23 01:32:08
	5	Producto 3	9697.98	Producto generado automaticamente	14	producto.jpg	1	10	0	2026-06-23 01:32:08
	6	Producto 4	6652.05	Producto generado automaticamente	1	producto.jpg	1	10	0	2026-06-23 01:32:08
	7	Producto 5	8318.61	Producto generado automaticamente	74	producto.jpg	1	3	0	2026-06-23 01:32:08
	8	Producto 6	13883.88	Producto generado automaticamente	66	producto.jpg	1	8	0	2026-06-23 01:32:08
	9	Producto 7	9107.48	Producto generado automaticamente	60	producto.jpg	1	4	0	2026-06-23 01:32:08
	10	Producto 8	1733.72	Producto generado automaticamente	14	producto.jpg	1	6	0	2026-06-23 01:32:08
	11	Producto 9	5795.71	Producto generado automaticamente	96	producto.jpg	1	9	0	2026-06-23 01:32:08
	12	Producto 10	4846.01	Producto generado automaticamente	81	producto.jpg	1	3	0	2026-06-23 01:32:08
	13	Producto 11	14196.88	Producto generado automaticamente	63	producto.jpg	1	5	0	2026-06-23 01:32:08
	14	Producto 12	7908.02	Producto generado automaticamente	91	producto.jpg	1	2	0	2026-06-23 01:32:08
	15	Producto 13	2326.28	Producto generado automaticamente	96	producto.jpg	1	5	0	2026-06-23 01:32:08
	16	Producto 14	10585.83	Producto generado automaticamente	70	producto.jpg	1	6	0	2026-06-23 01:32:08
	17	Producto 15	12056.99	Producto generado automaticamente	99	producto.jpg	1	8	0	2026-06-23 01:32:08
	18	Producto 16	11741.77	Producto generado automaticamente	37	producto.jpg	1	7	0	2026-06-23 01:32:08
	19	Producto 17	6803.07	Producto generado automaticamente	84	producto.jpg	1	1	0	2026-06-23 01:32:08
	20	Producto 18	10217.84	Producto generado automaticamente	100	producto.jpg	1	2	0	2026-06-23 01:32:08

Consultas avanzadas e índices

Se implementaron consultas utilizando distintas características del lenguaje SQL. En primer lugar, se realizaron consultas con JOIN para obtener información combinada entre productos y categorías, así como entre pedidos y usuarios.

También se desarrollaron consultas con GROUP BY y HAVING para identificar usuarios con una determinada cantidad de pedidos y una consulta con subconsulta para obtener productos cuyo precio se encontraba por encima del promedio general. Con el propósito de mejorar el rendimiento, se crearon índices sobre atributos frecuentemente utilizados en búsquedas y relaciones. Posteriormente se analizaron los planes de ejecución mediante la sentencia EXPLAIN, comprobando la utilización de dichos índices.

Además, se creó la vista denominada vw_pedidos_completos para facilitar el acceso a información consolidada de los pedidos realizados.

Capturas de pantalla de consultas avanzadas:

```
3      -- CONSULTA 1 - MOSTRAR PRODUCTOS CON SU CATEGORÍA
4
5  •   SELECT
6      p.id,
7      p.nombre AS producto,
8      p.precio,
9      c.nombre AS categoria
10     FROM producto p
11     INNER JOIN categoria c
12     ON p.categoria_id = c.id;

14     -- CONSULTA 2 - PEDIDOS CON INFORMACIÓN DEL USUARIO
15
16  •   SELECT
17      p.id,
18      p.fecha,
19      p.estado,
20      p.total,
21      CONCAT(u.nombre, ' ', u.apellido) AS usuario
22     FROM pedido p
23     INNER JOIN usuario u
24     ON p.usuario_id = u.id;

26     -- CONSULTA 3 - USUARIOS CON MÁS DE 3 PEDIDOS
27
28  •   SELECT
29      u.id,
30      CONCAT(u.nombre, ' ', u.apellido) AS usuario,
31      COUNT(p.id) cantidad_pedidos
32     FROM usuario u
33     INNER JOIN pedido p
34     ON u.id = p.usuario_id
35     GROUP BY u.id
36     HAVING COUNT(p.id) > 3;
```

```

38 -- CONSULTA 4 - PRODUCTOS CON PRECIO MAYOR AL PROMEDIO
39
40 • SELECT
41     nombre,
42     precio
43 FROM producto
44 WHERE precio >
45 (
46     SELECT AVG(precio)
47     FROM producto
48 );
49
52 • CREATE VIEW vw_pedidos_completos AS
53 SELECT
54     p.id,
55     p.fecha,
56     p.estado,
57     p.forma_pago,
58     p.total,
59     CONCAT(u.nombre, ' ', u.apellido) AS cliente
60 FROM pedido p
61 INNER JOIN usuario u
62 ON p.usuario_id = u.id;
63
64 • SELECT * FROM vw_pedidos_completos;

```

Capturas de pantalla de los índices:

```

3 -- CREAMOS ÍNDICES
4
5 • CREATE INDEX idx_producto_nombre ON producto(nombre);
6 • CREATE INDEX idx_producto_precio ON producto(precio);
7 • CREATE INDEX idx_pedido_usuario ON pedido(usuario_id);
-

```

29 • `SELECT p.nombre, c.nombre FROM producto p INNER JOIN categoria c ON p.categoria_id=c.id;`

	nombre	nombre
Producto 7	Bebidas	
Producto 37	Bebidas	
Producto 3	Cafeteria	
Producto 4	Cafeteria	
Producto 41	Cafeteria	
Producto 44	Cafeteria	
Producto 46	Cafeteria	
Producto 5	Empanadas	
Producto 10	Empanadas	
Producto 27	Empanadas	
Producto 28	Empanadas	
Producto 31	Empanadas	
Producto 49	Empanadas	
Producto 6	Ensaladas	
Producto 15	Ensaladas	
Producto 22	Ensaladas	
Producto 29	Ensaladas	

Producto 29	Ensaladas
Producto 42	Ensaladas
Hamburgue...	Hamburgu...
Producto 12	Hamburgu...
Producto 18	Hamburgu...
Producto 23	Hamburgu...
Producto 34	Hamburgu...
Producto 36	Hamburgu...
Producto 40	Hamburgu...
Producto 43	Hamburgu...
Producto 1	Helados
Producto 9	Helados
Producto 50	Helados
Producto 16	Pastas
Producto 19	Pastas
Producto 21	Pastas
Producto 45	Pastas
Producto 47	Pastas
Producto 48	Pastas
Muzzarella	Pizzas
Producto 17	Pizzas
Producto 20	Pizzas
Producto 24	Pizzas
Producto 25	Pizzas
Producto 32	Pizzas
Producto 33	Pizzas
Producto 38	Pizzas
Producto 11	Postres
Producto 13	Postres
Producto 30	Postres

Producto 2	Sandwiches
Producto 8	Sandwiches
Producto 14	Sandwiches
Producto 26	Sandwiches
Producto 35	Sandwiches
Producto 39	Sandwiches

```
35 • SELECT * FROM vw_pedidos_completos;
```

id	fecha	estado	forma_pago	total	cliente
1	2026-06-23	PENDIENTE	TARJETA	20500.00	Augusto Perez
40	2026-01-07	PENDIENTE	EFFECTIVO	0.00	Augusto Perez
321	2026-05-15	TERMINADO	TRANSFERENCIA	0.00	Augusto Perez
1334	2026-05-04	CANCELADO	EFFECTIVO	0.00	Augusto Perez
1807	2026-01-24	TERMINADO	TARJETA	0.00	Augusto Perez
163	2026-02-18	CONFIRMADO	TARJETA	0.00	Maria Lopez
1727	2026-02-07	PENDIENTE	TARJETA	0.00	Maria Lopez
53	2025-10-08	TERMINADO	EFFECTIVO	0.00	Nombre50 Apellido1

```
18 -- explain
```

```
19 • EXPLAIN SELECT * FROM producto WHERE precio BETWEEN 5000 AND 10000;
```

```
20
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	producto	NULL	range	idx_producto_precio	idx_producto_precio	5	NULL	18	100.00	Using index condition

```
24 -- explain
```

```
25 • EXPLAIN SELECT p.id, u.nombre FROM pedido p INNER JOIN usuario u ON p.usuario_id=u.id;
```

```
26
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	p	NULL	index	idx_pedido_usuario	idx_pedido_usuario	8	NULL	2501	100.00	Using index
1	SIMPLE	u	NULL	eq_ref	PRIMARY	PRIMARY	8	foodstore_db.p.usuario_id	1	100.00	Using index

Seguridad e integridad

Se implementaron distintos mecanismos para garantizar la seguridad e integridad de los datos. Creamos un usuario con privilegios limitados, otorgándole únicamente permisos necesarios para operar sobre la base de datos.

Además, se definieron vistas con el objetivo de ocultar información sensible, evitando exponer datos como contraseñas. También se realizaron pruebas de integridad, verificando el funcionamiento de las restricciones UNIQUE y de las claves foráneas, comprobando que el sistema impide la inserción de registros inconsistentes.

Finalmente, se implementó un procedimiento almacenado para realizar búsquedas de usuarios mediante parámetros, reduciendo riesgos asociados a posibles inyecciones SQL.

```
12 -- verificamos permisos
```

```
13 • SHOW GRANTS FOR 'operador_foodstore'@'localhost';
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Grants for operador_foodstore@localhost			
GRANT USAGE ON *.* TO `operador...			
GRANT SELECT, INSERT, UPDATE O...			

```
40 -- violación de UNIQUE donde intentamos insertar un mail repetido
```

```
41 • INSERT INTO usuario(nombre, apellido, mail, celular, contrasena, rol) VALUES ('Pedro', 'Gomez', 'juan@gmail.com', '3425000000', '1234', 'USUARIO');
```

104 02:31:22 INSERT INTO usuario(nombre, apellido, mail, celular, contrasena, rol) VALUES ('Pedro', 'Gomez', 'juan@gm... Error Code: 1062. Duplicate entry 'juan@gmail.com' for key 'usuario.uk_mail'

```
43 -- violación de FK
```

```
44 • INSERT INTO producto(nombre, precio, descripcion, stock, imagen, disponible, categoria_id) VALUES ('Producto Error', 1000, 'Prueba', 10, 'foto.jpg', TRUE, 99999);
```

105 02:33:27 INSERT INTO producto(nombre, precio, descripcion, stock, imagen, disponible, categoria_id) VALUES ('Producto Err... Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('foodstore_db'. 'producto', CONS...

```
26 • SELECT * FROM vw_usuarios_publicos;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

id	nombre	apellido	mail	celular	rol
1	Augusto	Perez	augustop@gmail.com	3425123456	USUARIO
2	Maria	Lopez	marial@gmail.com	3425678912	ADMIN
3	Nombre50	Apellido1	usuario50_1@gmail.com	3425000000	USUARIO
4	Nombre49	Apellido1	usuario49_1@gmail.com	3425000000	USUARIO
5	Nombre48	Apellido1	usuario48_1@gmail.com	3425000000	USUARIO
6	Nombre47	Apellido1	usuario47_1@gmail.com	3425000000	USUARIO
7	Nombre46	Apellido1	usuario46_1@gmail.com	3425000000	USUARIO
8	Nombre45	Apellido1	usuario45_1@gmail.com	3425000000	USUARIO
9	Nombre44	Apellido1	usuario44_1@gmail.com	3425000000	USUARIO
10	Nombre43	Apellido1	usuario43_1@gmail.com	3425000000	USUARIO
11	Nombre42	Apellido1	usuario42_1@gmail.com	3425000000	USUARIO
12	Nombre41	Apellido1	usuario41_1@gmail.com	3425000000	USUARIO
13	Nombre40	Apellido1	usuario40_1@gmail.com	3425000000	USUARIO
14	Nombre39	Apellido1	usuario39_1@gmail.com	3425000000	USUARIO

37 • `SELECT * FROM vw_productos_disponibles;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	id	nombre	precio	stock
▶	1	Muzzarella	8500.00	20
	2	Hamburguesa Completa	12000.00	15
	3	Producto 1	8784.68	88
	4	Producto 2	7673.56	79
	5	Producto 3	9697.98	14
	6	Producto 4	6652.05	1
	7	Producto 5	8318.61	74
	8	Producto 6	13883.88	66
	9	Producto 7	9107.48	60
	10	Producto 8	1733.72	14
	11	Producto 9	5795.71	96
	12	Producto 10	4846.01	81
	13	Producto 11	14196.88	63
	14	Producto 12	7908.02	91

65 • `CALL buscar_usuario_mail('juan@gmail.com');`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	id	nombre	apellido	mail	rol
▶	1026	Pedro	Gomez	juan@gmail.com	USUARIO

Concurrencia y transacciones

Con el objetivo de analizar el comportamiento del sistema ante operaciones simultáneas, se realizaron distintas pruebas utilizando transacciones. Verificamos el funcionamiento de COMMIT, confirmando de forma permanente las modificaciones realizadas sobre los datos. También se realizaron pruebas con ROLLBACK, permitiendo deshacer cambios y restaurar el estado previo de la información.

Luego analizamos los niveles de aislamiento disponibles y se efectuaron pruebas de concurrencia utilizando múltiples conexiones al servidor MySQL. Durante dichas pruebas se observaron situaciones de bloqueo entre transacciones, evidenciando el control de concurrencia proporcionado por el motor InnoDB.

Finalmente, con herramientas de monitoreo pudimos visualizar el estado de las transacciones activas y las esperas producidas entre sesiones concurrentes.



Query 1

```

1  -- PARTE 5 - BLOQUEO
2  -- sesión A
3  • USE foodstore_db;
4
5  • START TRANSACTION;
6
7  • UPDATE producto
8  SET stock = stock - 1
9  WHERE id = 1;

```

Output

#	Time	Action	Message
✓ 1	19:15:30	USE foodstore_db	0 row(s) affected
✓ 2	19:15:30	START TRANSACTION	0 row(s) affected
✓ 3	19:15:30	UPDATE producto SET stock = stock - 1 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 4	19:16:00	SELECT @@autocommit LIMIT 0, 1000	1 row(s) returned

Query 1

```

1  -- PARTE 5 - BLOQUEO
2  -- sesión B
3  • USE foodstore_db;
4
5  • UPDATE producto
6  SET stock = stock - 1
7  WHERE id = 1;

```

4 5 19:19:54 UPDATE producto SET stock = stock - 1 WHERE id = 1 Running...

4 17 19:26:07 UPDATE producto SET stock = stock - 1 WHERE id = 1 Running...

9 • COMMIT;

✓ 60 19:42:04 COMMIT 0 row(s) affected

15 • ROLLBACK;

✓ 61 19:42:43 ROLLBACK 0 row(s) affected

Conclusiones

A lo largo del desarrollo del trabajo práctico se aplicaron los principales conceptos estudiados en la materia. Se diseñó un modelo relacional normalizado y se implementaron mecanismos de integridad, seguridad y control de concurrencia. Asimismo, se realizaron consultas avanzadas y se utilizaron índices para optimizar el acceso a la información. Las pruebas realizadas permitieron comprobar el correcto funcionamiento del sistema y evidenciaron la importancia de mantener la consistencia de los datos mediante restricciones y transacciones.

Por último, podemos decir que la realización de este trabajo permitió consolidar conocimientos sobre diseño, implementación y administración de bases de datos relacionales utilizando MySQL.

Bibliografía

- Coronel, C. y Morris, S. (2019). Bases de Datos. Diseño, implementación y administración. Cengage Learning.
- Elmasri, R. y Navathe, S. (2016). Fundamentos de Sistemas de Bases de Datos. Pearson.
- MySQL Documentation. Oracle Corporation. Disponible en: <https://dev.mysql.com/doc/>
- Silberschatz, A., Korth, H. y Sudarshan, S. (2011). Fundamentos de Bases de Datos. McGraw-Hill.
- Material teórico de la cátedra Bases de Datos I.

Anexo IA

Durante el desarrollo del presente trabajo se utilizó inteligencia artificial como herramienta de apoyo para la consulta de conceptos, generación de ejemplos y asistencia en la elaboración de scripts SQL y documentación. La utilización de estas herramientas no reemplazó el proceso de análisis y comprensión de los temas desarrollados, sino que se usó como complemento para la organización del trabajo y la resolución de dudas puntuales. Las pruebas, verificaciones y adaptaciones realizadas sobre el código fueron ejecutadas y validadas por el autor del trabajo utilizando MySQL Workbench. Entre las herramientas utilizadas se encuentra ChatGPT (OpenAI), empleado para la generación de ejemplos y asistencia en la redacción de documentación técnica.